GitHub Watcher — Setup Guide

A free bot that posts your students' GitHub activity (commits, branches, pull requests, issues…) in your class Discord, with daily/weekly reports, deadlines and a teacher dashboard.

- **100% free**: Cloudflare Workers (free plan) + a GitHub App + Discord.
- **Students authorize themselves** with one link. They pick which repositories to share (private ones too) and can remove access anytime.
- **No server to run.** Everything lives in one file: `worker.js`.
- Time needed: about **30 minutes**. Do the steps in order.

What you get

Feature	Where
Live activity (pushes, branches, PRs, issues, failed CI…)	One forum thread per student (or one channel)
Daily ranking table (score, commits, lines, PRs)	Main channel, every working day
Weekly summary with trend	Main channel, every Friday
Inactive students alert (pings them)	Main channel
Commit quality checks (vague messages, tiny / huge commits, node_modules)	In each activity message
Deadlines with reminders (3 days, 1 day, final summary)	Main channel
Teacher dashboard (students, history, deadlines, settings, light/dark)	`YOUR-URL/admin?key=…`
AI review of a student's code vs commit messages and the brief (optional, only you see it)	Dashboard → student panel

> In this guide, **YOUR-URL** means your Worker address, e.g. `https://github-watcher.yourname.workers.dev` (no `/` at the end).

Step 1 — Cloudflare Worker

1. Create a free account: https://dash.cloudflare.com/sign-up
2. **Workers & Pages** → **Create** → **Start with Hello World** → name it `github-watcher` → **Deploy**.
3. Copy your Worker address. This is **YOUR-URL**.
4. Click **Edit code**, delete everything, paste all of `worker.js`, click **Deploy**.

Step 2 — Storage (KV)

1. **Storage & Databases** → **Workers KV** → **Create** → name `github-watcher-db`.
2. Open your Worker → **Bindings** → **Add binding** → **KV namespace**:
   - Variable name: `DB` (capital letters)
   - KV namespace: `github-watcher-db`

Step 3 — Daily report schedule (Cron)

Your Worker → **Settings** → **Trigger Events** → **Add** → **Cron Triggers** → enter:

```
15 17 * * *
```

This runs every day at **17:15 UTC**. Change it to your time **in UTC** (e.g. 18:00 in UTC+1 → `0 17 * * *`). If you change it, also add the variable `REPORT_TIME_UTC` with the same time (e.g. `17:00`), see Step 8.

Step 4 — Discord channels and webhooks

You need **2 webhooks** in your class Discord server:

A. Main channel (reports, alerts, deadlines)
Text channel (e.g. `#github-activity`) → ⚙️ **Edit Channel** → **Integrations** → **Webhooks** → **New Webhook** → **Copy Webhook URL**.
→ This is `DISCORD_WEBHOOK_URL`.

B. Forum channel (one thread per student — recommended)
Create Channel → type **Forum** (e.g. `github-students`) → ⚙️ **Edit Channel** → **Integrations** → **Webhooks** → **New Webhook** → **Copy Webhook URL**.
→ This is `DISCORD_FORUM_WEBHOOK_URL`. (Optional: if you skip it, all activity goes to the main channel.)

Step 5 — Discord app (login + bot)

1. Go to https://discord.com/developers/applications → **New Application** (e.g. `GitHub Watcher`).
2. **OAuth2** page:
   - Copy **Client ID** → `DISCORD_CLIENT_ID`
   - **Reset Secret** → copy **Client Secret** → `DISCORD_CLIENT_SECRET`
   - **Redirects** → **Add Redirect** → `YOUR-URL/discord/callback` → **Save Changes**
3. **Bot** page (shows students' real Discord names and names their threads):
   - **Reset Token** → copy it → `DISCORD_BOT_TOKEN`
   - Turn **Public Bot** off (optional).
4. Invite the bot to your server — open this link (replace `CLIENT_ID`), pick your server, **Authorize**:

```
https://discord.com/oauth2/authorize?client_id=CLIENT_ID&scope=bot&permissions=17179870208
```

It only gets **View Channels** + **Manage Threads**. It does not read messages.

Step 6 — GitHub App

Go to https://github.com/settings/apps/new and fill in:

Field	Value
GitHub App name	e.g. `MyClass Watcher` (students see this, must be unique)
Homepage URL	`YOUR-URL`
Callback URL	`YOUR-URL/github/callback`
Request user authorization (OAuth) during installation	❌ unchecked
Setup URL	`YOUR-URL/github/setup`
Redirect on update	✅ checked
Webhook → Active	✅ checked
Webhook URL	`YOUR-URL/github/webhook`
Webhook secret	any long random text — keep it → `GITHUB_WEBHOOK_SECRET`

Repository permissions → **Read-only** for: **Actions**, **Contents**, **Issues**, **Pull requests**. (Metadata is read-only automatically.) Nothing else.

Subscribe to events: Create, Delete, Push, Pull request, Pull request review, Pull request review comment, Issues, Issue comment, Commit comment, Release, Repository, Fork, Workflow run.

Where can this GitHub App be installed? → **Any account** → **Create GitHub App**.

Then on the app page:

1. Copy **Client ID** → `GITHUB_CLIENT_ID`
2. **Generate a new client secret** → copy → `GITHUB_CLIENT_SECRET`
3. Your app link is `https://github.com/apps/your-app-name`. The last part is the slug → `GITHUB_APP_SLUG`
4. Scroll down → **Private keys** → **Generate a private key**. A `.pem` file downloads. Open it with Notepad and copy **everything** (including the `-----BEGIN…` and `-----END…` lines) → `GITHUB_PRIVATE_KEY` (used to count lines in each commit)

Step 7 — Secrets in Cloudflare

Your Worker → **Settings** → **Variables and Secrets** → **Add** → Type **Secret**, one per line:

Name	Value
`DISCORD_WEBHOOK_URL`	Step 4A
`DISCORD_FORUM_WEBHOOK_URL`	Step 4B
`DISCORD_CLIENT_ID`	Step 5
`DISCORD_CLIENT_SECRET`	Step 5
`DISCORD_BOT_TOKEN`	Step 5
`GITHUB_APP_SLUG`	Step 6
`GITHUB_CLIENT_ID`	Step 6
`GITHUB_CLIENT_SECRET`	Step 6
`GITHUB_WEBHOOK_SECRET`	Step 6
`GITHUB_PRIVATE_KEY`	Step 6 (content of the .pem file)
`ADMIN_KEY`	a long password you invent, for your dashboard

Click **Deploy**. Names must be written exactly like this (capital letters).

Step 8 — Optional settings

Add these as **Text** variables only if you want to change the defaults:

Name	Default	Meaning
`REPORT_TIME_UTC`	`17:15`	Must match your cron time (Step 3). A "day" runs from this time to the same time next day
`SKIP_DAYS`	`sat,sun`	Days without a daily report (`none` = every day)
`WEEKLY_DAY`	`fri`	Day of the weekly summary
`INACTIVE_DAYS`	`3`	Working days with no activity before a student is pinged
`TINY_LINES`	`2`	Commits with this many changed lines or fewer are "tiny" (not scored)
`HUGE_LINES`	`2000`	Commits with this many lines or more are "huge" (not scored)
`IGNORE_EVENTS`	—	Events never posted, e.g. `fork,issue_comment`

Optional — AI review (free). Your Worker → **Bindings** → **Add binding** → **Workers AI** → variable name `AI` → **Deploy**. That's all: Cloudflare's own AI, no key, about 15–20 reviews per day for free (resets every day at 00:00 UTC). A Groq key (https://console.groq.com/keys, secret `GROQ_API_KEY`) is optional: if set, Groq is tried first and Cloudflare AI is the backup; if Groq refuses your Worker, it is skipped automatically for a few hours. Optional variables: `GROQ_MODEL`, `WORKERS_AI_MODEL`.

Step 9 — Check and share

1. Open your dashboard: `YOUR-URL/admin?key=YOUR_ADMIN_KEY` → **Settings** → **Setup check**. Everything should be green.
2. Post and pin this in your Discord:

> 📡 Connect your GitHub to share your activity here: **YOUR-URL**

Students click it → log in with Discord → install the GitHub App (choose repos) → ✅ they appear in the channel. Their next push is posted.

Using the dashboard

Open `YOUR-URL/admin?key=YOUR_ADMIN_KEY` (bookmark it, don't share it).

- **Overview** — class register (14 days), scores, students who need attention, deadlines.
- **Students** — search, filter, sort. Click a student for history, repos, "Change name", "Stop tracking".
- **Deadlines** — create a deadline (name, due date, "Count pushes from"), see who pushed, post status in Discord.
- **AI review** (student panel):
    1. In **Deadlines**, open a deadline → **Add a brief** → **Import brief page (.html)** (the brief page saved from Simplonline, e.g. with the SingleFile extension; click "voir le détail" first so all competencies are on the page). Name, brief text and competencies with levels are filled in (e.g. `C1N1, C3N1, CT1N1, CT5N1`: **C** = technical, **CT** = transversal competency, **N** = level). You can also paste the brief and type the codes. The repo name is optional (only a preference).
    2. Open a student → **AI review** → the running brief is preselected → **Analyze** (about 1 minute).
    3. You get: **sure checks** (pages, menu, semantic tags, `<h1>`, `alt`, SEO, form, map, responsive and the screen sizes of the brief, animation, README, site online, W3C validator, commit messages, planning link in the README (GitHub Projects, Trello… when CT1/CT2 is targeted) — Tailwind and Bootstrap are understood; checks the brief does not ask for are left out), **each user story / bonus / deliverable / criterion** with its proof, a **suggestion per competency** (your decision stays on Simplonline), the **commits vs messages**, and a **feedback draft** written from the sure checks, which you can edit and copy.
    4. Every line (requirement, sure check, competency) has a switch: **Valid** (green) or **Not valid** (red). The AI and the checks set the first state; click to change it. A requirement proved by a check moves with that check. Things the code cannot show (oral, meeting) start as Valid: your call.

5. **Mise en situation** (optional): click **Propose a challenge**. The AI draws, at random, a user story of the brief (valid or not in the review) and writes one small task at the level of the brief (level 1: 10 min, level 2: 15 min, level 3: 20 min): a small slice, never a whole page, using only the brief's technologies (no JavaScript in an HTML/CSS brief, no new feature, no bonus). A story the learner did is a "show me you master it" task; a story not done is a "show me you understand how to do it" task. The learner does it in front of you. **Copy for the learner**, then choose **Valid / Partly valid / Not valid** and add a comment if you want. **Another challenge** draws a new one.
6. Click **Generate comment**: the comment for the learner is written from your final switches and the challenge result (and saved with them). Edit it if needed, then **Copy comment**. Bonus lines are optional (performance): praised when done, never asked for. The comment always has a **Git / GitHub** part: number and regularity of commits, clarity and style of the messages (feat:, fix:…), and branches / merges when the learner uses them.
7. Students keep their report and their planning link in the README: CT5 is judged on the README, CT1/CT2 on the planning link found there. Other transversal points are seen orally, so the AI never claims them "validated" without proof.

   Nothing is posted in Discord. You can add your own evaluation criteria per competency level in `REF_CRITERIA` in `worker.js` (keys `C1` … `C8`, and `T1` … `T9` for CT1…CT9).

- **Settings** — light/dark theme, refresh real names, send reports now (preview / daily / weekly), hide repos, setup check.

Scoring: Score = real commits + pull requests + other activity. Tiny commits, docs-only commits and huge commits (e.g. `node_modules`) are shown with a warning but not scored. Vague messages (`update`, `fix`, `.`) are flagged.

Students: privacy and removal

- Students choose exactly which repositories are shared. Access is **read-only**.
- To stop: GitHub → **Settings** → **Applications** → **Installed GitHub Apps** → your app → **Uninstall**.
- A student can also ask you to hide a single repo (dashboard → Settings → Hidden repos).
- Activity statistics are kept for **15 days** only.
- If you use the AI review, the analyzed code is sent to the AI service (Cloudflare and/or Groq) and the home page HTML to the W3C validator. Tell your students.

Security tips

- **Never share** your secrets: webhook URLs, bot token, client secrets, private key, `ADMIN_KEY`. Anyone with a webhook URL can post in your channel.
- If something leaks: reset it (Discord → Reset Token / delete webhook; GitHub → new client secret / new private key), then update the secret in Cloudflare and **Deploy**.
- Turn on 2-factor authentication on Cloudflare, GitHub and Discord.

Troubleshooting

Problem	Fix
`redirect_uri invalid` on Discord login	Step 5: the redirect must be exactly `YOUR-URL/discord/callback`
Student connected but nothing is posted	Check the GitHub App webhook URL + secret (Step 6). In the App → **Advanced** → **Recent Deliveries** you can see errors
Error 1031 / preview error	Use the normal `workers.dev` URL, not the preview URL
Names are GitHub usernames	Add `DISCORD_BOT_TOKEN` and invite the bot (Step 5), then Settings → refresh names
Threads are not renamed	The bot needs **Manage Threads** in the forum channel
"Line counting" not green	Add `GITHUB_PRIVATE_KEY` (whole .pem content)
AI review: "Cloudflare AI free daily limit reached"	About 15–20 reviews a day are free. Wait until tomorrow (00:00 UTC)
AI review: "small AI model" warning	The big model was busy or refused: click Analyze again later
AI review: Groq error 403 "Access denied"	Groq blocks your Worker's network. Add the Workers AI binding `AI`; Groq is then skipped automatically
No daily report	Check the cron (Step 3) and that today is not a skip day

Free plan limits: fine for a class of about 30–40 students. One Worker = one class/Discord server. For another class, repeat the setup with another Worker, GitHub App and channels.